

Unit 6 : Basic Network Management

6.1 IP address configuration: settings in a network. TCP/IP Network address,

TCP/IP Configuration files, Network Interfaces and Routes: ifconfig, route,

ping, netstat, tcpdump commands.

6.2 DHCP Configuration: Configuring DHCP Client and Server, Dynamic Address, Fixed Addresses.

6.3 NIS, NFS, SAMBA introduction.

6.4 Firewall and Internet Security: Limiting Network Services, Designing Firewall.

6.5 Network Intrusion Detection: Host based Intrusion Detection Software using tripwire any relevant utility.

6.3 : NIS, NFS, SAMBA introduction.

6.3 A : NIS

The Network Information Service, or NIS (initially called YP or yellow pages), is a mainframe-client index service convention for circulating server configuration information, for example, client and host names between PCs on a PC network. Sun Microsystems built up the NIS; the innovation is authorized to essentially all other Unix merchants. Since British Telecom PLC claimed the name “Yellow Pages” as an enlisted brand name in the United Kingdom for its paper-based, business phone catalog, Sun changed the name of its framework to NIS, however, all the orders capacities actually start with “yp”. A NIS/YP framework keeps up and disseminates a focal index of the client and gathering information, hostnames, email pseudonyms, and other content-based tables of information in a PC network. For instance, in a typical UNIX climate, the rundown of clients for ID is put in/and so forth/passwd and mystery verification hashes in/and so on/shadow. NIS includes another “worldwide” client list which is utilized for recognizing clients on any customer of the NIS area. Administrators can arrange NIS to serve secret key information to outside cycles to verify clients utilizing different variants of the Unix crypt(3) hash calculations. In any case, in such cases, any NIS(0307) customer can recover the whole secret phrase information base for disconnected investigation. Kerberos was intended to deal with confirmation in a safer way.

We utilize the Linux NIS mainframe (Network Information Service) for sharing basic information put away in level documents between frameworks on a

network. It is frequently ideal to have a mutual archive, (for example, NIS) for putting away clients and gatherings information as opposed to putting away them in level documents like/and so forth/passwd. So what is the advantage of that? By making such documents accessible through the NIS worker, that would permit any distant NIS customer machine to access or question the information in these mutual records and use them as expansions to the nearby forms.

NIS isn't for sharing records. We can share any even document which in any event has one section with an extraordinary worth by means of NIS like/and so on/services record. The primary advantage of utilizing the NIS worker is that you keep your information and records, and spread any updates to all clients. A few clients, particularly Windows clients, may think this is a kind of Active Directory like service. The Linux NIS worker is more established than Active Directory and not a reproduce for it.

The standard model for networking protocols and distributed applications is the International Organization for Standardization (ISO) seven-layer model shown in Table 1-1.

Table 1-1. The ISO seven-layer model

Layer	Name	Physical Layer
7	Application	NFS and NIS
6	Presentation	XDR
5	Session	RPC
4	Transport	TCP or UDP
3	Network	IP
2	Data Link	Ethernet
1	Physical	CAT-5

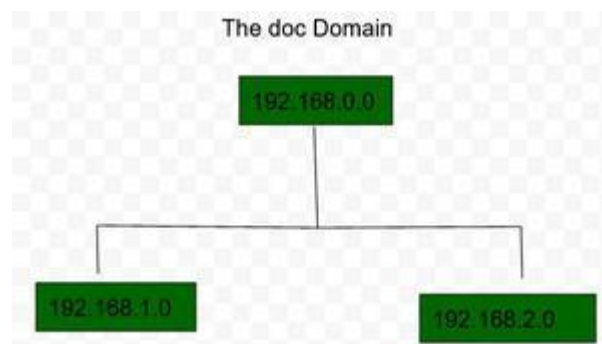
NIS Introduction:

By running NIS, the framework administrator can disperse administrative information bases, called maps, among an assortment of mainframes (ace and slaves). The administrator can refresh those information bases from a brought together area in a programmed and solid design to guarantee that all customers share a similar naming service information predictably all through the network. NIS was grown freely of DNS and has a marginally extraordinary core interest. While DNS centers around making correspondence less difficult by utilizing machine names rather than mathematical IP addresses, NIS centers around making network administration more sensible by giving unified command over an assortment of network information. NIS stores information about machine names and addresses, yet additionally about clients, the network itself, and network services. This assortment of network information is alluded to as the NIS namespace.

Note — In certain settings machine names are alluded to have had names or machine names. This conversation utilizes a machine, yet some screen messages or NIS map names may utilize host or machine.

NIS Architecture:

NIS utilizes a customer worker plan. NIS mainframes offer types of assistance to NIS customers. The primary workers are called ace workers, and for unwavering quality, they have reinforcement or slave workers. Both ace and slave workers utilize the NIS information recovery programming and both store NIS maps. NIS utilizes domains to mastermind the machines, clients, and networks in its namespace. Nonetheless, it doesn't utilize an area chain of command; a NIS namespace is level.



A NIS domain can't be associated straightforwardly to the Internet utilizing just NIS. Notwithstanding, associations that need to utilize NIS and furthermore be associated with the Internet can consolidate NIS with DNS. You can utilize NIS to deal with all neighborhood information and use DNS for the Internet to have queries. NIS gives a sending service that advances have queries to DNS if the information can't be found in a NIS map. The Solaris working climate additionally permits you to set up the `nsswitch.conf` record with the goal that has query demands go just to DNS, or to DNS and

afterward NIS on the off chance that not found by DNS, or to NIS and afterward DNS if not found by NIS.

NIS Machine Types: There are three kinds of NIS machines:

- Ace mainframe
- Slave mainframe
- Customers of NIS mainframes

Any machine can be a NIS customer, however, just machines with plates ought to be NIS workers, either ace or slave. Workers are likewise customers, ordinarily of themselves.

NIS Servers:

NIS mainframes come in two assortments, ace, and slave. The machine assigned as ace worker contains the arrangement of guides that the framework administrator makes and updates as important. Every NIS space must have one, and just one, ace worker, which can engender NIS refreshes with the least presentation debasement. We can assign extra NIS workers in the domain as slave workers. A slave worker has a total duplicate of the ace arrangement of NIS maps. At whatever point the ace worker maps are refreshed, the updates are proliferated among the slave workers. Slave workers can deal with any flood of solicitations from the ace worker, limiting “worker inaccessible” mistakes.

Ordinarily, the framework administrator assigns one ace worker for all NIS maps. Be that as it may, on the grounds that every individual NIS map has the machine name of the ace worker encoded inside it, you could assign various workers to go about as ace and slave workers for various guides. To limit disarray, assign a solitary worker as the ace for all the guides you make inside a solitary area. The models in this part expect that one worker is the ace for all guides in the space.

NIS Clients:

NIS customers run measures that demand information from maps on the workers. Customers don’t make a differentiation among ace and slave workers, since all NIS workers ought to have a piece of similar information.

NIS Elements:

The NIS naming service is made out of the accompanying components:

Domains

Guides

Daemons

Utilities

NIS Command Set

The NIS is an information base that contains a progression of tables. It makes tables from text records like/and so on/passwd,/and so forth/services, and some other plain documents. Each table may contain one section or more with a novel key on each line. You can consider it like any typical information base. You can question these tables in two different ways:

Posting the whole table

Pulling a particular passage via looking

At the point when a program solicitations to look for a client secret phrase subtleties, the customer checks the/and so on/passwd record to check on the off chance that the client doesn't exist there; the customer at that point approaches the NIS worker to look for it in the/and so on/passwd table from the NIS worker. You can utilize any of the services and applications that accompany the NIS mainframe:

ypserv: This service sits tight for inquiries and offers responses to NIS customers.

ypbind: This is the customer side of NIS.

ypxfrd: You can utilize this service for sending the NIS information bases from ace MIS workers to slave workers.

The NIS Domain: A NIS domain is an assortment of machines that share a typical arrangement of NIS maps. Every space has an area name and each machine sharing the basic arrangement of guides has a place with that area. Any machine can have a place with a given area, as long as there is a mainframe for that space's guides in a similar network. A NIS customer machine acquires its area name and ties to a NIS worker as a component of its boot cycle.

Five daemons are utilized in order to provide the NIS service:

Daemon	Function
ypserv	Server process
ypbind	Binding process
ypxfrd	High-speed map transfer
rpc.yppasswdd	NIS password update daemon
rpc.yppupdated	Modifies other maps such as <i>publickey</i>

A total of 9 utilities support the NIS service:

Utility	Function
makedbm	Makes dbm record for a NIS map
ypcat	Records information in a guide
ypinit	Fabricates and introduces a NIS information base and instates NIS customer's ypservers list
ypmatch	Finds a particular passage in a guide
yppoll	Gets a guide request number from a worker
yppush	Proliferates information from NIS ace to NIS slave worker
ypset	Sets binding to a specific mainframe
ypwhich	Records name of the NIS mainframe and alias interpretation table
ypxfr	Moves information from ace to slave NIS mainframe

Applications of NIS:

There are some valuable apparatuses that can assist you with dealing with the information in the information base.

- ypcat: You can utilize this to get information from the NIS worker by separating it from the NIS map.
- ypwhich: gets the name of the Linux NIS worker that is reacting to your solicitations.
- ypmatch: instead of snatching the whole guide, or you can look by key to get a particular section.

6.3 B :NFS

NFS is an abbreviation of the **Network File System**. It is a protocol of a distributed file system. This protocol was developed by the Sun Microsystems in the year of 1984.

It is an architecture of the client/server, which contains a client program, server program, and a protocol that helps for communication between the client and server.

It is that protocol which allows the users to access the data and files remotely over the network. Any user can easily implement the NFS protocol because it is an open standard. Any user can manipulate files as same as if they were on like other protocols. This protocol is also built on the ONC RPC system.

This protocol is mainly implemented on those computing environments where the centralized management of resources and data is critical. It uses the Transmission Control Protocol (TCP) and User Datagram Protocol (UDP) for accessing and delivering the data and files.

Features

Network File System is a protocol that works on all the networks of IP-based. It is implemented in that client/server application in which the server of NFS manages the authorization, authentication, and clients. This protocol is used with Apple Mac OS, Unix, and Unix-like operating systems such as Solaris, Linux, FreeBSD, AIX.

NFS (Network File System) is basically developed for sharing of **files** and **folders** between **Linux/Unix** systems. It allows you to mount your local file systems over a network and remote hosts to interact with them as they are mounted locally on the same system. With the help of **NFS**, we can set up file sharing between **Unix** to **Linux** system and **Linux** to **Unix** system.

Benefits of NFS

- **NFS** allows local access to remote files.
- It uses standard **client/server** architecture for file sharing between all ***nix** based machines.
- With **NFS** it is not necessary that both machines run on the same **OS**.
- With the help of **NFS** we can configure **centralized storage** solutions.
- Users get their **data** irrespective of physical location.
- No manual **refresh** needed for new files.
- Newer version of **NFS** also supports **acl**, **pseudo** root mounts.
- Can be secured with **Firewalls** and **Kerberos**.

NFS Services

Its a **System V-launched** service. The **NFS** server package includes three facilities, included in the **portmap** and **nfs-utils** packages.

- **portmap** : It maps calls made from other machines to the correct **RPC** service (not required with **NFSv4**).
- **nfs**: It translates remote **file sharing** requests into requests on the local file system.
- **rpc.mountd**: This service is responsible for **mounting** and **unmounting** of file systems.

Important Files for NFS Configuration

- **/etc/exports** : Its a main configuration file of **NFS**, all exported **files** and **directories** are defined in this file at the **NFS Server** end.
- **/etc/fstab** : To mount a **NFS directory** on your system across the **reboots**, we need to make an entry in **/etc/fstab**.
- **/etc/sysconfig/nfs** : Configuration file of **NFS** to control on which port **rpc** and other services are **listening**.

Setup and Configure NFS Mounts on Linux Server

To setup **NFS** mounts, we'll be needing at least two **Linux/Unix** machines. Here in this tutorial, I'll be using two servers.

- **NFS Server**: nfsserver.example.com with IP-**192.168.0.100**
- **NFS Client** : nfsclient.example.com with IP-**192.168.0.101**

Installing NFS Server and NFS Client

We need to install **NFS** packages on our **NFS Server** as well as on **NFS Client** machine. We can install it via "**yum**" (**Red Hat** Linux) and "**apt-get**" (**Debian** and **Ubuntu**) package installers.

```
[root@nfsserver ~]# yum install nfs-utils nfs-utils-lib
```

```
[root@nfsserver ~]# yum install portmap (not required with NFSv4)
```

```
[root@nfsserver ~]# apt-get install nfs-utils nfs-utils-lib
```

Now start the **services** on both machines.

```
[root@nfsserver ~]# /etc/init.d/portmap start
```

```
[root@nfsserver ~]# /etc/init.d/nfs start
```

```
[root@nfsserver ~]# chkconfig --level 35 portmap on
```

```
[root@nfsserver ~]# chkconfig --level 35 nfs on
```

After installing packages and starting services on both the machines, we need to configure both the machines for file sharing.

Setting Up the NFS Server

First we will be configuring the **NFS** server.

Configure Export directory

For sharing a directory with **NFS**, we need to make an entry in "**/etc/exports**" configuration file. Here I'll be creating a new directory named "**nfsshare**" in "/" partition to share with **client server**, you can also share an already existing directory with **NFS**.

```
[root@nfsserver ~]# mkdir /nfsshare
```

Now we need to make an entry in "**/etc/exports**" and **restart** the services to make our directory shareable in the network.

```
[root@nfsserver ~]# vi /etc/exports
```

```
/nfsshare 192.168.0.101(rw,sync,no_root_squash)
```

In the above example, there is a directory in / partition named "**nfsshare**" is being shared with client IP "**192.168.0.101**" with **read** and **write (rw)** privilege, you can also use **hostname** of the client in the place of **IP** in above example.

NFS Options

Some other options we can use in "**/etc/exports**" file for file sharing is as follows.

1. **ro**: With the help of this option we can provide **read only access** to the shared files i.e **client** will only be able to **read**.
2. **rw**: This option allows the **client server** to both **read** and **write** access within the shared directory.
3. **sync**: Sync confirms requests to the shared directory only once the **changes** have been committed.

4. **no_subtree_check**: This option prevents the **subtree** checking. When a shared directory is the subdirectory of a larger file system, **nfs** performs scans of every directory above it, in order to verify its permissions and details. Disabling the **subtree** check may increase the reliability of **NFS**, but reduce **security**.
5. **no_root_squash**: This phrase allows **root** to **connect** to the designated directory.

For more options with **"/etc/exports"**, you are recommended to read the **man pages** for **export**.

Setting Up the NFS Client

After configuring the **NFS** server, we need to **mount** that shared directory or partition in the **client** server.

Mount Shared Directories on NFS Client

Now at the **NFS client** end, we need to **mount** that directory in our server to access it locally. To do so, first we need to find out that shares available on the remote server or NFS Server.

```
[root@nfsclient ~]# showmount -e 192.168.0.100
```

Export list for 192.168.0.100:

```
/nfsshare 192.168.0.101
```

Above command shows that a directory named **"nfsshare"** is available at **"192.168.0.100"** to share with your server.

Mount Shared NFS Directory

To **mount** that shared **NFS** directory we can use following mount command.

```
[root@nfsclient ~]# mount -t nfs 192.168.0.100:/nfsshare /mnt/nfsshare
```

The above command will mount that shared directory in **"/mnt/nfsshare"** on the client server. You can verify it following command.

```
[root@nfsclient ~]# mount | grep nfs
```

```
sunrpc on /var/lib/nfs/rpc_pipefs type rpc_pipefs (rw)
```

```
nfsd on /proc/fs/nfsd type nfsd (rw)
```

```
192.168.0.100:/nfsshare on /mnt type nfs (rw,addr=192.168.0.100)
```

The above mount command mounted the **nfs shared directory** on to **nfs client** temporarily, to mount an NFS directory **permanently** on your system across the **reboots**, we need to make an entry in **"/etc/fstab"**.

```
[root@nfsclient ~]# vi /etc/fstab
```

Add the following new line as shown below.

```
192.168.0.100:/nfsshare /mnt nfs defaults 0 0
```

Test the Working of NFS Setup

We can test our **NFS server setup** by creating a **test file** on the server end and check its availability at **nfs client** side or vice-versa.

At the nfsserver end

I have created a new text file named "**nfstest.txt**" in that shared directory.

```
[root@nfsserver ~]# cat > /nfsshare/nfstest.txt
```

This is a test file to test the working of NFS server setup.

At the nfsclient end

Go to that shared directory in **client server** and you'll find that shared file without any manual refresh or service restart.

```
[root@nfsclient]# ll /mnt/nfsshare
```

```
total 4
```

```
-rw-r--r-- 1 root root 61 Sep 21 21:44 nfstest.txt
```

```
root@nfsclient ~]# cat /mnt/nfsshare/nfstest.txt
```

This is a test file to test the working of NFS server setup.

Removing the NFS Mount

If you want to **unmount** that shared directory from your server after you are done with the file sharing, you can simply **unmount** that particular directory with "**umount**" command. See this example below.

```
root@nfsclient ~]# umount /mnt/nfsshare
```

You can see that the mounts were removed by then looking at the filesystem again.

```
[root@nfsclient ~]# df -h -F nfs
```

You'll see that those shared directories are not available any more.

Important commands for NFS

Some more important commands for **NFS**.

1. **showmount -e** : Shows the available **shares** on your local machine
2. **showmount -e <server-ip or hostname>**: Lists the available **shares** at the **remote** server
3. **showmount -d** : Lists all the **sub directories**
4. **exportfs -v** : Displays a list of shares **files** and **options** on a server
5. **exportfs -a** : Exports all shares listed in **/etc/exports**, or given name
6. **exportfs -u** : Unexports all shares listed in **/etc/exports**, or given name
7. **exportfs -r** : Refresh the server's list after modifying **/etc/exports**

This is it with **NFS mounts** for now, this was just a start, I'll come up with more option and features of **NFS** in our future articles. Till then, Stay connected with **Tecmint.com** for more exciting and interesting tutorials in future. Do leave your **comments and suggestions** below in the comment box.

6.3 C : SAMBA introduction

Since computers seldom work as isolated systems, it is to be expected that as a system administrator or engineer, you know how to set up and maintain a network with multiple types of servers.

In this article and in the next of this series we will go through the essentials of setting up **Samba** and **NFS** servers with Windows/Linux and Linux clients, respectively.

To set up file servers in corporate or enterprise environments where you are likely to find different operating systems and types of devices Samba server can be used.

Step 1: Installing Samba Server

Our current testing environment consists of two **RHEL 7 boxes** and one **Windows 8** machine, in that order:

1. **Samba / NFS server [box1 (RHEL 7): 192.168.0.18],**
2. **Samba client #1 [box2 (RHEL 7): 192.168.0.20]**
3. **Samba client #2 [Windows 8 machine: 192.168.0.106]**



```
root@box1:~# cat /etc/redhat-release
Red Hat Enterprise Linux Server release 7.1 (Maipo)
root@box1:~#

root@box2:~# cat /etc/redhat-release
Red Hat Enterprise Linux Server release 7.1 (Maipo)
root@box2:~#
```

Testing Setup for Samba

On **box1**, install the following packages:

```
# yum update && yum install samba samba-client samba-common
```

On **box2**:

```
# yum update && yum install samba samba-client samba-common cifs-utils
```

Once the installation is complete, we're ready to configure our share.

Step 2: Setting Up File Sharing Through Samba

One of the reason why **Samba** is so relevant is because it provides file and print services to **SMB/CIFS** clients, which causes those clients to see the server as if it was a Windows system (I must admit I tend to get a little emotional while writing about this topic as it was my first setup as a new Linux system administrator some years ago).

Adding system users and setting up permissions and ownership

To allow for group collaboration, we will create a group named **finance** with two users (**user1** and **user2**) with useradd command and a directory **/finance** in **box1**.

We will also change the group owner of this directory to **finance** and set its permissions to **0770** (read, write, and execution permissions for the owner and the group owner):

```
# groupadd finance
# useradd user1
# useradd user2
# usermod -a -G finance user1
# usermod -a -G finance user2
# mkdir /finance
# chmod 0770 /finance
# chgrp finance /finance
```

Step 3: Configuring SELinux and Firewall

In preparation to configure **/finance** as a Samba share, we will need to either disable **SELinux** or set the proper **boolean** and security context values as follows (otherwise, SELinux will prevent clients from accessing the share):

```
# setsebool -P samba_export_all_ro=1 samba_export_all_rw=1
# getsebool -a | grep samba_export
# semanage fcontext -at samba_share_t "/finance(/.*)?"
# restorecon /finance
```

In addition, we must ensure that Samba traffic is allowed by the firewalld.

```
# firewall-cmd --permanent --add-service=samba
# firewall-cmd --reload
```

Step 4: Configure Samba Share

Now it's time to dive into the configuration file **/etc/samba/smb.conf** and add the section for our share: we want the members of the **finance** group to be able to browse the contents of **/finance**, and save / create files or subdirectories in it (which by default will have their permission bits set to **0770** and **finance** will be their group owner):

smb.conf

```
[finance]
comment=Directory for collaboration of the company's finance team
browsable=yes
path=/finance
public=no
valid users=@finance
write list=@finance
writeable=yes
create mask=0770
Force create mode=0770
force group=finance
```

Save the file and then test it with the **testparm** utility. If there are any errors, the output of the following command will indicate what you need to fix. Otherwise, it will display a review of your Samba server configuration:

```
[root@box1 ~]# testparm
Load smb config files from /etc/samba/smb.conf
rlimit_max: increasing rlimit_max (1024) to minimum Windows limit (16384)
Processing section "[homes]"
Processing section "[printers]"
Processing section "[finance]"
Loaded services file OK.
Server role: ROLE_STANDALONE
Press enter to see a dump of your service definitions
```

Test Samba Configuration

Should you want to add another share that is open to the public (meaning without any authentication whatsoever), create another section in **/etc/samba/smb.conf** and under the new share's name copy the section above, only changing **public=no** to **public=yes** and not including the valid users and write list directives.

Step 5: Adding Samba Users

Next, you will need to add **user1** and **user2** as Samba users. To do so, you will use the **smbpasswd** command, which interacts with Samba's internal database. You will be prompted to enter a password that you will later use to connect to the share:

```
# smbpasswd -a user1
```

```
# smbpasswd -a user2
```

Finally, restart **Samba**, enable the service to start on boot, and make sure the share is actually available to network clients:

```
# systemctl start smb
```

```
# systemctl enable smb
```

```
# smbclient -L localhost -U user1
```

```
# smbclient -L localhost -U user2
```

```
[root@box1 ~]# smbclient -L localhost -U user1
Enter user1's password:
Domain=[MYGROUP] OS=[Unix] Server=[Samba 4.1.12]

      Sharename      Type      Comment
      -----
      finance        Disk      Directory for collaboration of the company's finance team
      IPC$           IPC       IPC Service (Samba Server Version 4.1.12)
      user1          Disk      Home Directories
Domain=[MYGROUP] OS=[Unix] Server=[Samba 4.1.12]

      Server          Comment
      -----
      Workgroup        Master
      -----
[root@box1 ~]# smbclient -L localhost -U user2
Enter user2's password:
Domain=[MYGROUP] OS=[Unix] Server=[Samba 4.1.12]

      Sharename      Type      Comment
      -----
      finance        Disk      Directory for collaboration of the company's finance team
      IPC$           IPC       IPC Service (Samba Server Version 4.1.12)
      user2          Disk      Home Directories
Domain=[MYGROUP] OS=[Unix] Server=[Samba 4.1.12]

      Server          Comment
      -----
      Workgroup        Master
      -----
[root@box1 ~]#
```

Verify Samba Share

At this point, the Samba file server has been properly installed and configured. Now it's time to test this setup on our **RHEL 7** and **Windows 8** clients.

Step 6: Mounting the Samba Share in Linux

First, make sure the Samba share is accessible from this client:

```
# smbclient -L 192.168.0.18 -U user2
```

```
[root@box2 ~]# smbclient -L 192.168.0.18 -U user2
Enter user2's password:
Domain=[MYGROUP] OS=[Unix] Server=[Samba 4.1.12]

      Sharename      Type      Comment
      -----
      IPC$           IPC       IPC Service (Samba Server Version 4.1.12)
      finance        Disk      Directory for collaboration of the company's finance team
      user2          Disk      Home Directories
Domain=[MYGROUP] OS=[Unix] Server=[Samba 4.1.12]

      Server          Comment
      -----
      Workgroup        Master
      -----
[root@box2 ~]#
```

Mount Samba Share on Linux
(repeat the above command for user1)

As any other storage media, you can mount (and later unmount) this network share when needed:

```
# mount //192.168.0.18/finance /media/samba -o username=user1
```

```
[root@box2 ~]# mount | grep finance → The Samba share has not been mounted yet
[root@box2 ~]# mount //192.168.0.18/finance /media/samba -o username=user1
Password for user1@//192.168.0.18/finance: → Enter the password for user1
[root@box2 ~]# mount | grep finance
//192.168.0.18/finance on /media/samba type cifs (rw,relatime,vers=1.0,cache=strict,username=user1,do
posixpaths,serverino,acl,rsize=1048576,wsiz=65536,actimeo=1) → The Samba share is now mounted
[root@box2 ~]#
```

Mount Samba Network Share

(where **/media/samba** is an existing directory)

or permanently, by adding the following entry in **/etc/fstab** file:

fstab

```
//192.168.0.18/finance /media/samba cifs
```

```
credentials=/media/samba/.smbcredentials,defaults 0 0
```

Where the hidden file **/media/samba/.smbcredentials** (whose permissions and ownership have been set to **600** and **root:root**, respectively) contains two lines that indicate the username and password of an account that is allowed to use the share:

.smbcredentials

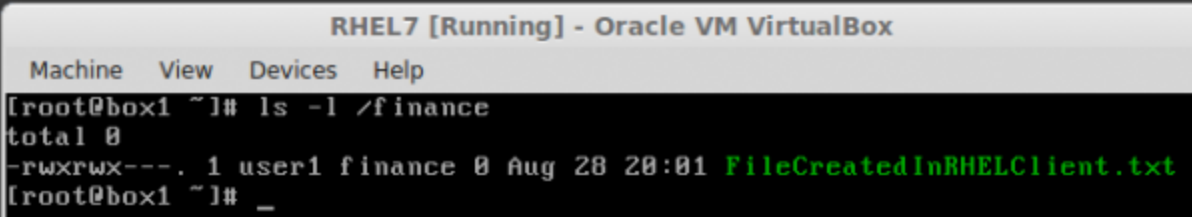
```
username=user1
```

```
password=PasswordForUser1
```

Finally, let's create a file inside **/finance** and check the permissions and ownership:

```
# touch /media/samba/FileCreatedInRHELClient.txt
```

```
[root@box2 ~]# touch /media/samba/FileCreatedInRHELClient.txt
[root@box2 ~]#
```

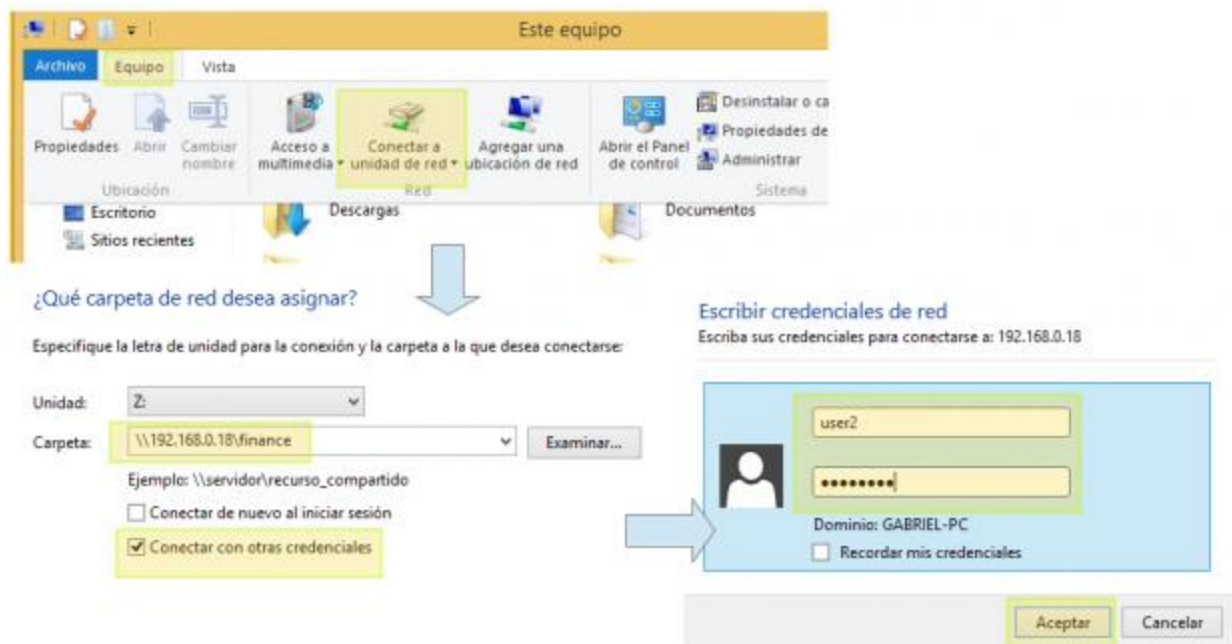


Create File in Samba Share

As you can see, the file was created with **0770** permissions and ownership set to **user1:finance**.

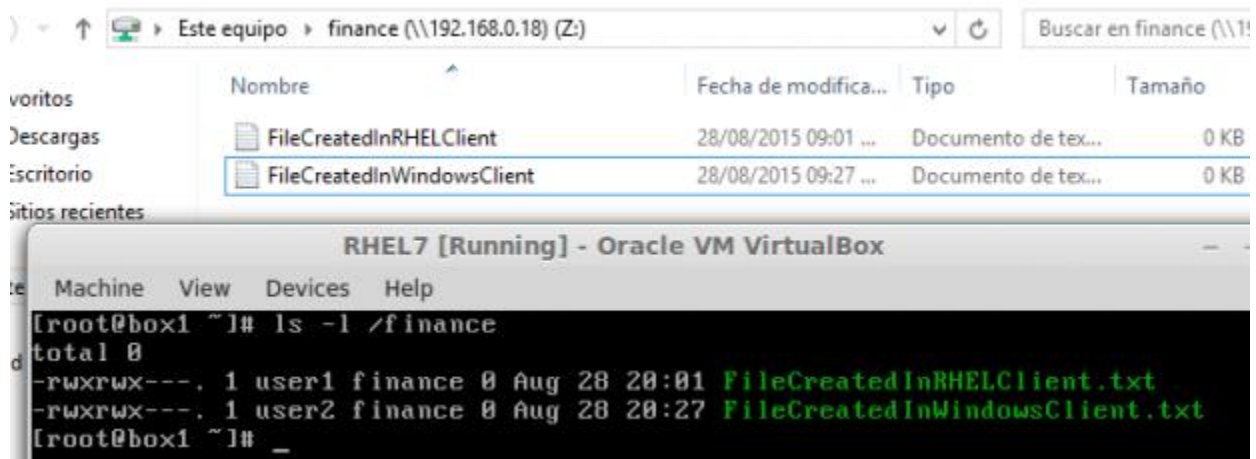
Step 7: Mounting the Samba Share in Windows

To mount the Samba share in Windows, go to **My PC** and choose **Computer**, then **Map** network drive. Next, assign a letter for the drive to be mapped and check Connect using different credentials (the screenshots below are in Spanish, my native language):



Mount Samba Share in Windows

Finally, let's create a file and check the permissions and ownership:



Create Files on Windows Samba Share

ls -l /finance

This time the file belongs to **user2** since that's the account we used to connect from the Windows client.